Agentic economy

Register your first AI agent

Copy page

Register AI agents with onchain identity, build reputation, and verify credentials using ERC-8004 on Arc Testnet.

This quickstart guides you through registering an AI agent using the **ERC-8004** standard on Arc Testnet. You'll create developer-controlled wallets, register your agent's identity, record reputation events, and verify credentials. Select the tab that matches your preferred setup.

ERC-8004 contracts on Arc testnet

Contract	Address
IdentityRegistry	`0x8004A818BFB912233c491871b3d84c89A494BD9e`
ReputationRegistry	`0x8004B663056A597Dffe9eCcC1965A193B7388713`
ValidationRegistry	`0x8004Cb1BF31DAf7788923b405b754f57acEB4272`

Circle Wallets Viem

Prerequisites

Before you begin, make sure you have:

1. A **Circle Developer Console** account
2. An API key created in the Console: **Keys → Create a key → API key → Standard Key**
3. Your **Entity Secret registered**

Step 1. Set up your project

Create a project directory, install dependencies, and configure your environment.

1.1. Create the project and install dependencies

Node.js Python

```
mkdir erc8004-quickstart
cd erc8004-quickstart
npm init -y
npm pkg set type=module
npm pkg set scripts.start="tsx --env-file=.env index.ts"

npm install @circle-fin/developer-controlled-wallets viem
npm install --save-dev tsx typescript @types/node
```

1.2. Configure TypeScript (optional)

This step is optional. It helps prevent missing types in your IDE or editor.

Create a `tsconfig.json` file:

```
npx tsc --init
```

Then, update the `tsconfig.json` file:

```
cat <<'EOF' > tsconfig.json
{
  "compilerOptions": {
    "target": "ESNext",
    "module": "ESNext",
    "moduleResolution": "bundler",
    "strict": true,
    "types": ["node"]
  }
}
EOF
```

1.3. Set environment variables

Create a `.env` file in the project directory and add your Circle credentials:

```
CIRCLE_API_KEY=YOUR_API_KEY
CIRCLE_ENTITY_SECRET=YOUR_ENTITY_SECRET
```

Where `YOUR_API_KEY` is your Circle Developer API key and `YOUR_ENTITY_SECRET` is your registered Entity Secret.

The `npm run start` command loads these variables from `.env` using Node.js native env-file support.

> Prefer editing `.env` files in your IDE or editor so credentials are not leaked to your shell history.

Step 2. Create developer-controlled wallets

In this step, you create two Arc Testnet dev-controlled wallets for the ERC-8004 flow. One wallet owns the agent and the other records reputation. If you already have two Arc Testnet dev-controlled wallets for this flow, skip to **Step 3**. Per ERC-8004, agent owners cannot record reputation for their own agents to prevent self-dealing.

The Step 2 through 7 code snippets explain the flow in smaller pieces. They are not cumulative and will not run if pasted together. To run the full workflow end to end, use the **complete script** at the end of this tutorial.

index.ts index.py

```
import { initiateDeveloperControlledWalletsClient } from "@circle-fin/developer
                                                        ›
const circleClient = initiateDeveloperControlledWalletsClient({
  apiKey: process.env.CIRCLE_API_KEY!,
  entitySecret: process.env.CIRCLE_ENTITY_SECRET!,
});

const walletSet = await circleClient.createWalletSet({
  name: "ERC8004 Agent Wallets",
});

const walletsResponse = await circleClient.createWallets({
  blockchains: ["ARC-TESTNET"],
  count: 2,
  walletSetId: walletSet.data?.walletSet?.id ?? "",
  accountType: "SCA",
});

const ownerWallet = walletsResponse.data?.wallets?.[0]!;
const validatorWallet = walletsResponse.data?.wallets?.[1]!;

console.log(`Owner:     ${ownerWallet.address}`);
console.log(`Validator: ${validatorWallet.address}`);
```

Step 3. Prepare agent metadata

Create a JSON file with metadata for your agent. The structure below is an example you can adapt for your use case. ERC-8004 registration stores a metadata URI, but the JSON fields at that URI are application-defined unless your integration follows a separate metadata convention.

agent-metadata.json

```
{
  "name": "DeFi Arbitrage Agent v1.0",
  "description": "Autonomous trading agent for cross-DEX arbitrage on Arc",
  "image": "ipfs://QmAgentAvatarHash...",
  "agent_type": "trading",
  "capabilities": [
    "arbitrage_detection",
    "liquidity_monitoring",
    "automated_execution"
  ],
  "version": "1.0.0"
}
```

Upload to IPFS using **Pinata**, **NFT.Storage**, **Web3.Storage** or your preferred IPFS tool. You'll receive an IPFS URI like `ipfs://QmYourHash...` .

> For this quickstart, you can skip uploading and use the example URI: `ipfs://bafkreibdi6623n3xpf7ymk62ckb4bo75o3qemwkpfvp5i25j66itxvsoei`

Step 4. Register your agent identity

Call `register(metadataURI)` on the IdentityRegistry to mint an identity NFT for your agent.

index.ts index.py

```
const IDENTITY_REGISTRY = "0x8004A818BFB912233c491871b3d84c89A494BD9e";

const METADATA_URI =
  process.env.METADATA_URI ||
  "ipfs://bafkreibdi6623n3xpf7ymk62ckb4bo75o3qemwkpfvp5i25j66itxvsoei";

const registerTx = await circleClient.createContractExecutionTransaction({
  walletAddress: ownerWallet.address!,
  blockchain: "ARC-TESTNET",
  contractAddress: IDENTITY_REGISTRY,
  abiFunctionSignature: "register(string)",
  abiParameters: [METADATA_URI],
  fee: { type: "level", config: { feeLevel: "MEDIUM" } },
});

// Poll until confirmed
let txHash: string | undefined;
for (let i = 0; i < 30; i++) {
  await new Promise((r) => setTimeout(r, 2000));
  const { data } = await circleClient.getTransaction({
    id: registerTx.data?.id!,
  });
  if (data?.transaction?.state === "COMPLETE") {
    txHash = data.transaction.txHash;
    break;
  }
  if (data?.transaction?.state === "FAILED")
    throw new Error("Registration failed");
}

console.log(`Registered: https://testnet.arcscan.app/tx/${txHash}`);
```

> With Circle Gas Station, your application sponsors the transaction fees. On Arc, gas is approximately 0.006 USDC-TESTNET per transaction.

Step 5. Retrieve your agent ID

Query the `Transfer` event from the IdentityRegistry to find the token ID minted for your agent.

index.ts index.py

```
import { createPublicClient, http, parseAbiItem, getContract } from "viem";
import { arcTestnet } from "viem/chains";

const publicClient = createPublicClient({
  chain: arcTestnet,
  transport: http(),
});

const latestBlock = await publicClient.getBlockNumber();
const blockRange = 10000n; // RPC limit: eth_getLogs is often capped at 10,000
const fromBlock = latestBlock > blockRange ? latestBlock - blockRange : 0n;

const transferLogs = await publicClient.getLogs({
  address: IDENTITY_REGISTRY,
  event: parseAbiItem(
    "event Transfer(address indexed from, address indexed to, uint256 indexed
  ),
  args: { to: ownerWallet.address as `0x${string}` },
  fromBlock,
  toBlock: latestBlock,
});

if (transferLogs.length === 0) {
  throw new Error("No Transfer events found — registration may have failed");
}

const agentId = transferLogs[transferLogs.length - 1].args.tokenId!.toString()

const identityContract = getContract({
  address: IDENTITY_REGISTRY,
  abi: [
    {
      name: "ownerOf",
      type: "function",
      stateMutability: "view",
      inputs: [{ name: "tokenId", type: "uint256" }],
      outputs: [{ name: "", type: "address" }],
    },
    {
      name: "tokenURI",
      type: "function",
```

```
        stateMutability: "view",
        inputs: [{ name: "tokenId", type: "uint256" }],
        outputs: [{ name: "", type: "string" }],
      },
    ],
    client: publicClient,
  });

  const owner = await identityContract.read.ownerOf([BigInt(agentId)]);
  const tokenURI = await identityContract.read.tokenURI([BigInt(agentId)]);

  console.log(`Agent ID: ${agentId}`);
  console.log(`Owner: ${owner}`);
  console.log(`Metadata: ${tokenURI}`);
```

Your AI agent now has a unique onchain identity.

Step 6. Record reputation

Build your agent's reputation by recording feedback. Use the **validator wallet** — per ERC-8004, agent owners cannot record reputation for their own agents.

index.ts index.py

```
import { keccak256, toHex } from "viem";

const REPUTATION_REGISTRY = "0x8004B663056A597Dffe9eCcC1965A193B7388713";

const tag = "successful_trade";
const feedbackHash = keccak256(toHex(tag));

const reputationTx = await circleClient.createContractExecutionTransaction({
  walletAddress: validatorWallet.address!,
  blockchain: "ARC-TESTNET",
  contractAddress: REPUTATION_REGISTRY,
  abiFunctionSignature:
    "giveFeedback(uint256,int128,uint8,string,string,string,string,bytes32)",
  abiParameters: [agentId, "95", "0", tag, "", "", "", feedbackHash],
  fee: { type: "level", config: { feeLevel: "MEDIUM" } },
});

// Poll until confirmed (same pattern as Step 4)
```

Production scoring: This quickstart hardcodes `score: 95` for demonstration. In production, calculate scores dynamically based on agent behavior. For example, `score = loanRepaidOnTime ? 100 : 20` for lending protocols, or `score = slippagePct < 1 ? 95 : 60` for trading platforms.

The ReputationRegistry stores attestations from external observers who witnessed the agent's actions. Your application logic calculates scores based on outcomes, then records them onchain.

Step 7. Request and verify validation

The ERC-8004 ValidationRegistry uses a two-step request/response flow. The **agent owner** requests validation from a validator, then the **validator** submits a response.

index.ts index.py

```
const VALIDATION_REGISTRY = "0x8004Cb1BF31DAf7788923b405b754f57acEB4272";

const requestURI = "ipfs://bafkreiexamplevalidationrequest";
const requestHash = keccak256(
  toHex(`kyc_verification_request_agent_${agentId}`),
);

// Owner requests validation
const validationReqTx = await circleClient.createContractExecutionTransaction(
  walletAddress: ownerWallet.address!,
  blockchain: "ARC-TESTNET",
  contractAddress: VALIDATION_REGISTRY,
  abiFunctionSignature: "validationRequest(address,uint256,string,bytes32)",
  abiParameters: [validatorWallet.address!, agentId, requestURI, requestHash],
  fee: { type: "level", config: { feeLevel: "MEDIUM" } },
});

// Poll until confirmed (same pattern as Step 4)

// Validator responds (100 = passed, 0 = failed)
const validationResTx = await circleClient.createContractExecutionTransaction(
  walletAddress: validatorWallet.address!,
  blockchain: "ARC-TESTNET",
  contractAddress: VALIDATION_REGISTRY,
  abiFunctionSignature:
    "validationResponse(bytes32,uint8,string,bytes32,string)",
  abiParameters: [
    requestHash,
    "100",
    "",
    "0x" + "0".repeat(64),
    "kyc_verified",
  ],
  fee: { type: "level", config: { feeLevel: "MEDIUM" } },
});

// Poll until confirmed, then verify:
const validationContract = getContract({
  address: VALIDATION_REGISTRY,
  abi: [
    {
```

```
        name: "getValidationStatus",
        type: "function",
        stateMutability: "view",
        inputs: [{ name: "requestHash", type: "bytes32" }],
        outputs: [
          { name: "validatorAddress", type: "address" },
          { name: "agentId", type: "uint256" },
          { name: "response", type: "uint8" },
          { name: "responseHash", type: "bytes32" },
          { name: "tag", type: "string" },
          { name: "lastUpdate", type: "uint256" },
        ],
      },
    ],
    client: publicClient,
  });

  type ValidationStatus = readonly [
    `0x${string}`,
    bigint,
    number,
    `0x${string}`,
    string,
    bigint,
  ];

  const [valAddr, , response, , tag] =
    (await validationContract.read.getValidationStatus([
      requestHash,
    ])) as ValidationStatus;

  console.log(`Validator: ${valAddr}`);
  console.log(`Response: ${response} (100 = passed)`);
  console.log(`Tag: ${tag}`);
```

Full agent registration script

The complete script below combines all the preceding steps into a single runnable file.

index.ts index.py

```
import { initiateDeveloperControlledWalletsClient } from "@circle-fin/develope
import {
  createPublicClient,
  http,
  parseAbiItem,
  getContract,
  keccak256,
```

··· See all 266 lines

Save it, then run:

Node.js Python

```
npm run start
```

> If you followed the Python workflow, run `deactivate` when you're done to exit the virtual environment.

Summary

After completing this quickstart, you've successfully:

- Created or prepared two Arc Testnet wallets for the ERC-8004 flow
- Registered an AI agent with a unique onchain identity (ERC-721 token)
- Recorded reputation feedback from an external validator
- Requested validation from a validator and verified the response onchain

Was this page helpful? Yes No

‹ Monitor contract events Create your first ERC-8183 job ›

By using the Arc Network Testnet, you agree to the Terms of Service and Privacy Policy.